The Journal of Open Source Software

dexamine: A Python package to parse decentralized exchange data from Ethereum

Magnus Hansson [1,2]

1 Stockholm University, Sweden **2** Swedish House of Finance, Sweden

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

Summary

Decentralized exchanges allow users to trade digital assets through programs executed on a blockchain. Their public transaction records provide a source of data for studying trading and liquidity provision, but these records require interpretation before they can be used in empirical research. `dexamine` is a Python package that converts Ethereum transaction records into structured observations of activity on Uniswap v2 and v3 (Adams et al., 2020, 2021). It extracts trades and changes in liquidity, identifies the tokens involved, and associates each event with its transaction costs and execution order. The package is intended for researchers studying price discovery, liquidity provision, and market microstructure in decentralized finance.

Statement of need

Research on decentralized exchanges requires data at the level of individual events. Trade prices and volumes alone do not describe the conditions under which a transaction executed. Its position within a block, execution fees, and the pool state following a trade are relevant to the analysis of market quality and arbitrage (Barbon & Ranaldo, 2026; Daian et al., 2020; Lehar & Parlour, 2025). A single transaction can also contain several trades or liquidity events, which must remain distinguishable in the resulting dataset.

Ethereum exposes blocks, transactions, and event logs through its JSON-RPC interface. Constructing a research dataset from these records requires protocol-specific decoding, resolution of token metadata, conversion of integer quantities into token units, and joins between events and execution metadata. Repeating these steps in individual research projects increases implementation effort and creates opportunities for inconsistent units, signs, and event ordering.

`dexamine` provides these transformations in a reusable library. Given a block number and transaction index, it retrieves the corresponding transaction, receipt, and block, then produces an observation for each supported event. Its scope is Uniswap v2 and v3 on Ethereum mainnet, with support for swaps and liquidity additions and removals represented by mint and burn events. Transaction discovery, data storage, and statistical analysis remain separate stages of the research workflow.

State of the field

Several existing tools provide components of this workflow. `web3.py` (web3.py contributors, 2024) offers Ethereum RPC access, contract calls, and ABI-based event decoding; `dexamine` uses it for contract metadata queries. Ethereum ETL (Medvedev & D5 contributors, 2018) exports blockchain records and selected token data. `cryo` (Paradigm, 2023) supports bulk extraction into tabular formats and event decoding from supplied signatures. These tools

provide general extraction capabilities, while constructing the Uniswap event representation described here still requires protocol-specific transformations and joins.

Graph Node (The Graph Protocol, 2018) supports application-specific indexing and GraphQL queries through subgraphs. It can be operated locally, and the information retained depends on the subgraph schema and mappings. It is therefore an alternative for persistent indexed queries, although it requires maintaining an indexing service. TrueBlocks (TrueBlocks Team, 2022) provides address-based transaction indexing and can supply transaction positions for subsequent parsing by `dexamine`.

The reason for a separate library is the combination of Uniswap event semantics and transaction metadata in a common research schema. Extending a generic RPC client with this schema would introduce assumptions specific to one application, while a subgraph would couple the transformations to an indexing service. `dexamine` instead reuses existing RPC and contract-access facilities and implements the protocol interpretation as a Python library that researchers can incorporate into their own data pipelines.

Software design

The implementation separates JSON-RPC retrieval, cached contract metadata, protocol parsers, and output construction. A reusable session batches requests and retains pool and token metadata across calls. Results are yielded incrementally, so event records need not accumulate in memory; the metadata cache grows with the number of distinct contracts encountered. This design supports processing transaction samples as well as larger datasets without requiring a database service.

The parsers account for differences in how the protocols report pool state. Uniswap v2 emits reserve updates in a preceding `Sync` event. The parser checks that the immediately preceding log is a `Sync` from the same pool and skips the associated event if this condition fails. Uniswap v3 swap logs report price, tick, and active liquidity directly. The parser derives virtual reserves from these quantities and reports them as missing when active liquidity is zero. Virtual reserves describe the local trading curve; they are not the pool's total token balances.

The output can retain the original node payloads alongside parsed events or provide one flat row per event. Flat records retain block, transaction, and log indices, as well as transaction hashes, to preserve the link to source records. Token amounts are scaled by their decimal precision. Derived quantities use floating-point arithmetic, so the normalized output is intended for empirical analysis rather than exact integer accounting. Gas consumption and fee fields describe the whole transaction and are repeated across its events; they do not allocate execution costs to individual trades.

Transaction destination labels use a bundled list of Uniswap router and periphery addresses. Other destinations and contract creation receive separate labels. This identifies the top-level destination, without reconstructing internal call paths or inferring trader identity, arbitrage, or maximal extractable value. Such attribution requires additional evidence and remains part of the researcher's analysis.

Reproducibility depends on the software version, source responses, and metadata. Pool and token metadata are queried at the latest block and cached, rather than resolved historically. Consequently, changes to token metadata can affect repeated runs even when historical event logs are unchanged. Retaining input responses and resolved metadata is advisable for replication. Historical state access is not required by this design, but the endpoint must serve the requested historical blocks and receipts and support the contract calls used for metadata resolution.

The Journal of Open Source Software

Research impact statement

dexamine has been used to construct datasets for *Price Discovery in Constant Product Markets* (Hansson, 2024), which studies trading and liquidity provision on Uniswap. This application motivates preserving execution order and pool state alongside trades and liquidity events. The package supplies data preparation; trader classification and econometric estimation belong to the associated research workflow.

The repository contains installation instructions, worked parsing examples, descriptions of output fields and limitations, and contribution and support guidance. Recorded JSON-RPC fixtures support offline tests of event decoding, signed quantities, reserve ordering, pool filtering, and missing values. An optional integration suite checks parsing against a live endpoint. Continuous integration runs the offline tests, formatting, linting, and strict type checking on Python 3.10 through 3.13. These materials allow prospective users to inspect the transformations and evaluate their suitability for other studies. The source and development history are maintained in the software repository.

AI usage disclosure

OpenAI Codex (GPT-6) assisted with restructuring and editing this manuscript, checking claims against the implementation and cited sources, and preparing a submission readiness review. Verification during this revision included the offline test suite and compilation of the manuscript. This disclosure covers the present revision; prior AI use and the author's review of AI-assisted material require confirmation before submission.

Acknowledgements

I thank the TrueBlocks and Erigon (Erigon Technologies, 2022) teams for assistance with node operation and indexing, and members of the Flashbots and Uniswap communities for discussions of the protocols.

References

Adams, H., Zinsmeister, N., & Robinson, D. (2020). *Uniswap v2 core*. Uniswap. https://uniswap.org/whitepaper.pdf

Adams, H., Zinsmeister, N., Salem, M., Keefer, R., & Robinson, D. (2021). *Uniswap v3 core*. Uniswap. https://uniswap.org/whitepaper-v3.pdf

Barbon, A., & Ranaldo, A. (2026). On the quality of cryptocurrency markets: Centralized vs. Decentralized exchanges. *Management Science*. https://doi.org/10.1287/mnsc.2024.07703

Daian, P., Goldfeder, S., Kell, T., Li, Y., Zhao, X., Bentov, I., Breidenbach, L., & Juels, A. (2020). Flash boys 2.0: Frontrunning in decentralized exchanges, miner extractable value, and consensus instability. *2020 IEEE Symposium on Security and Privacy (SP)*, 910–927. https://doi.org/10.1109/SP40000.2020.00040

Erigon Technologies. (2022). *Erigon: An Ethereum implementation on the efficiency frontier*. https://github.com/erigontech/erigon

Hansson, M. (2024). *Price discovery in constant product markets*. SSRN working paper. https://doi.org/10.2139/ssrn.4582649

Lehar, A., & Parlour, C. A. (2025). Decentralized exchange: The Uniswap automated market maker. *The Journal of Finance*, *80*(1), 321–374. https://doi.org/10.1111/jofi.13405

Medvedev, E., & D5 contributors. (2018). *Ethereum ETL: Export blockchain data to CSV or JSON files*. https://github.com/blockchain-etl/ethereum-etl

Paradigm. (2023). *cryo: Extract blockchain data to Parquet, CSV, JSON or Python dataframes*. https://github.com/paradigmxyz/cryo

The Graph Protocol. (2018). *Graph Node: An indexing and query layer for blockchain data*. https://github.com/graphprotocol/graph-node

TrueBlocks Team. (2022). *TrueBlocks: Lightweight indexing for any EVM-based blockchain*. https://github.com/TrueBlocks/trueblocks-core

web3.py contributors. (2024). *web3.py: A Python library for interacting with Ethereum*. https://web3py.readthedocs.io